

External Memory Interface

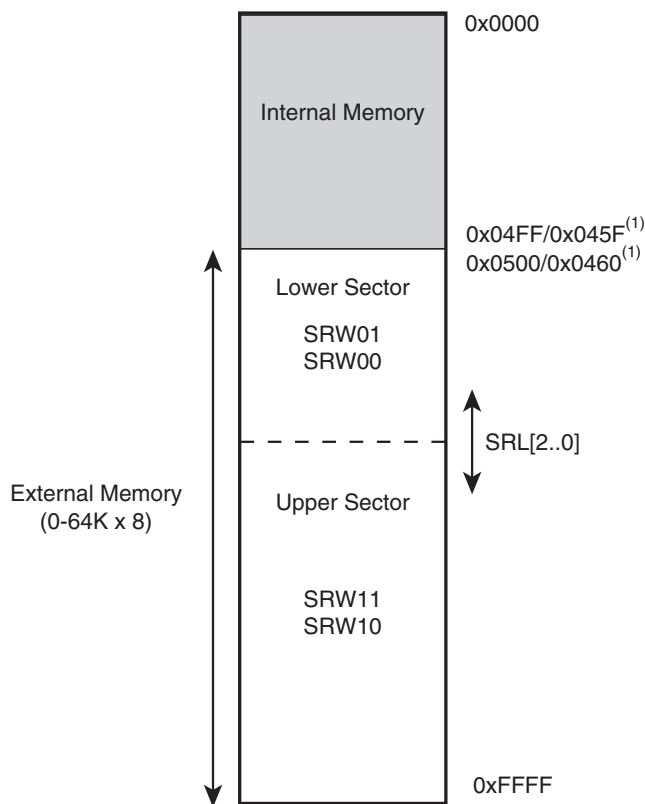
With all the features the External Memory Interface provides, it is well suited to operate as an interface to memory devices such as external SRAM and FLASH, and peripherals such as LCD-display, A/D, and D/A. The main features are:

- **Four Different Wait-state Settings (Including No Wait-state)**
- **Independent Wait-state Setting for Different External Memory Sectors (Configurable Sector Size)**
- **The Number of Bits Dedicated to Address High Byte is Selectable**
- **Bus Keepers on Data Lines to Minimize Current Consumption (Optional)**

Overview

When the eXternal MEMORY (XMEM) is enabled, address space outside the internal SRAM becomes available using the dedicated external memory pins (see [Figure 1 on page 2](#), [Table 29 on page 70](#), [Table 35 on page 75](#), and [Table 41 on page 81](#)). The memory configuration is shown in [Figure 11](#).

Figure 11. External Memory with Sector Select



Note: 1. Address depends on the ATmega161 compatibility Fuse. See [“SRAM Data Memory” on page 18](#) and [Figure 9 on page 19](#) for details.

Using the External Memory Interface

The interface consists of:

- AD7:0: Multiplexed low-order address bus and data bus
- A15:8: High-order address bus (configurable number of bits)
- ALE: Address latch enable
- $\overline{\text{RD}}$: Read strobe.
- $\overline{\text{WR}}$: Write strobe.

The control bits for the External Memory Interface are located in three registers, the MCU Control Register – MCUCR, the Extended MCU Control Register – EMCUCR, and the Special Function IO Register – SFIOR.

When the XMEM interface is enabled, it will override the settings in the Data Direction registers corresponding to the ports dedicated to the interface. For details about this port override, see the alternate functions in section “I/O-Ports” on page 63. The XMEM interface will autodetect whether an access is internal or external. If the access is external, the XMEM interface will output address, data, and the control signals on the ports according to Figure 13 (this figure shows the wave forms without wait-states). When ALE goes from high to low, there is a valid address on AD7:0. ALE is low during a data transfer. When the XMEM interface is enabled, also an internal access will cause activity on address-, data- and ALE ports, but the RD and WR strobes will not toggle during internal access. When the External Memory Interface is disabled, the normal pin and data direction settings are used. Note that when the XMEM interface is disabled, the address space above the internal SRAM boundary is not mapped into the internal SRAM. Figure 12 illustrates how to connect an external SRAM to the AVR using an octal latch (typically “74x573” or equivalent) which is transparent when G is high.

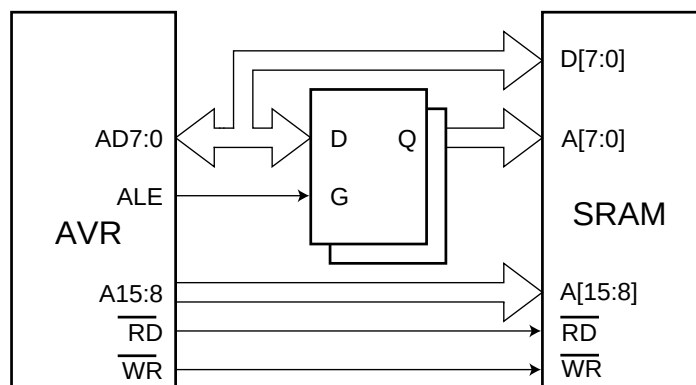
Address Latch Requirements

Due to the high-speed operation of the XRAM interface, the address latch must be selected with care for system frequencies above 8 MHz @ 4V and 4 MHz @ 2.7V. When operating at conditions above these frequencies, the typical old style 74HC series latch becomes inadequate. The external memory interface is designed in compliance to the 74AHC series latch. However, most latches can be used as long they comply with the main timing parameters. The main parameters for the address latch are:

- D to Q propagation delay (t_{pd}).
- Data setup time before G low (t_{su}).
- Data (address) hold time after G low (t_{th}).

The external memory interface is designed to guaranty minimum address hold time after G is asserted low of $t_h = 5$ ns (refer to t_{LAXX_LD}/t_{LLAXX_ST} in Table 114 to Table 121 on page 272). The D to Q propagation delay (t_{pd}) must be taken into consideration when calculating the access time requirement of the external component. The data setup time before G low (t_{su}) must not exceed address valid to ALE low (t_{AVLLC}) minus PCB wiring delay (dependent on the capacitive load).

Figure 12. External SRAM Connected to the AVR



Pull-up and Bus Keeper

The pull-up resistors on the AD7:0 ports may be activated if the corresponding Port register is written to one. To reduce power consumption in sleep mode, it is recommended to disable the pull-ups by writing the Port register to zero before entering sleep.

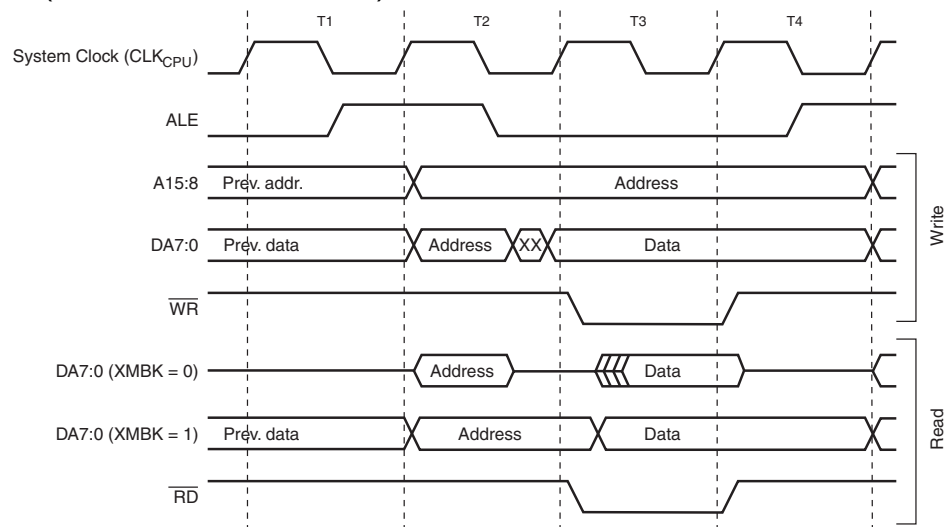
The XMEM interface also provides a bus keeper on the AD7:0 lines. The Bus Keeper can be disabled and enabled in software as described in “[Special Function IO Register – SFIOR](#)” on [page 32](#). When enabled, the Bus Keeper will keep the previous value on the AD7:0 bus while these lines are tri-stated by the XMEM interface.

Timing

External memory devices have various timing requirements. To meet these requirements, the ATmega162 XMEM interface provides four different wait-states as shown in [Table 3](#). It is important to consider the timing specification of the external memory device before selecting the wait-state. The most important parameters are the access time for the external memory in conjunction with the set-up requirement of the ATmega162. The access time for the external memory is defined to be the time from receiving the chip select/address until the data of this address actually is driven on the bus. The access time cannot exceed the time from the ALE pulse is asserted low until data must be stable during a read sequence ($t_{LLRL} + t_{RLRH} - t_{DVRH}$ in [Table 114](#) to [Table 121](#) on [page 272](#)). The different wait-states are set up in software. As an additional feature, it is possible to divide the external memory space in two sectors with individual wait-state settings. This makes it possible to connect two different memory devices with different timing requirements to the same XMEM interface. For XMEM interface timing details, please refer to [Figure 118](#) to [Figure 121](#), and [Table 114](#) to [Table 121](#).

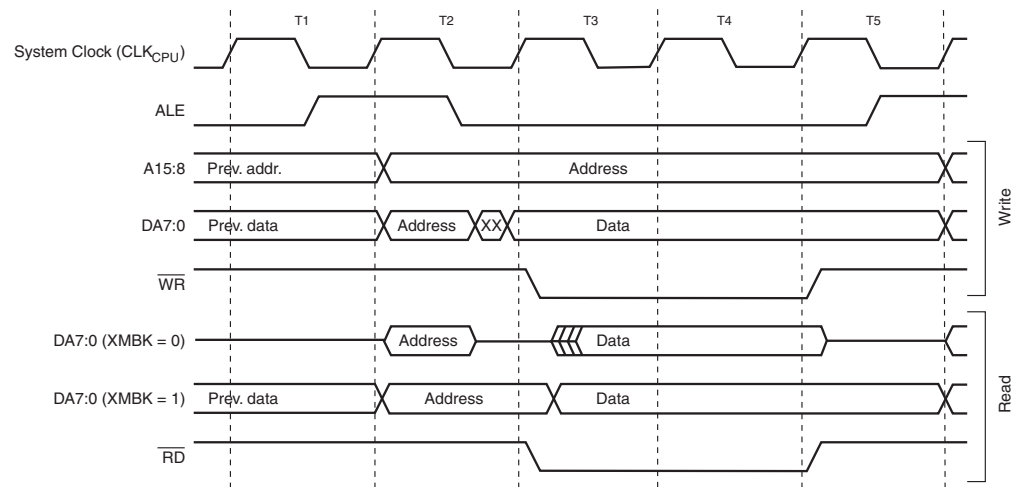
Note that the XMEM interface is asynchronous and that the waveforms in the figures below are related to the internal system clock. The skew between the internal and external clock (XTAL1) is not guaranteed (it varies between devices, temperature, and supply voltage). Consequently, the XMEM interface is not suited for synchronous operation.

Figure 13. External Data Memory Cycles without Wait-state
(SRWn1 = 0 and SRWn0 = 0)⁽¹⁾



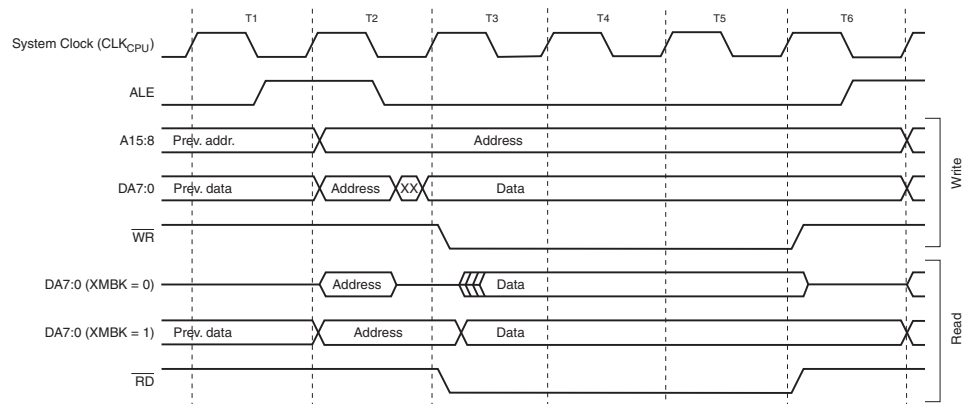
Note: 1. SRWn1 = SRW11 (upper sector) or SRW01 (lower sector), SRWn0 = SRW10 (upper sector) or SRW00 (lower sector).
The ALE pulse in period T4 is only present if the next instruction accesses the RAM (internal or external).

Figure 14. External Data Memory Cycles with $SRWn1 = 0$ and $SRWn0 = 1$ ⁽¹⁾



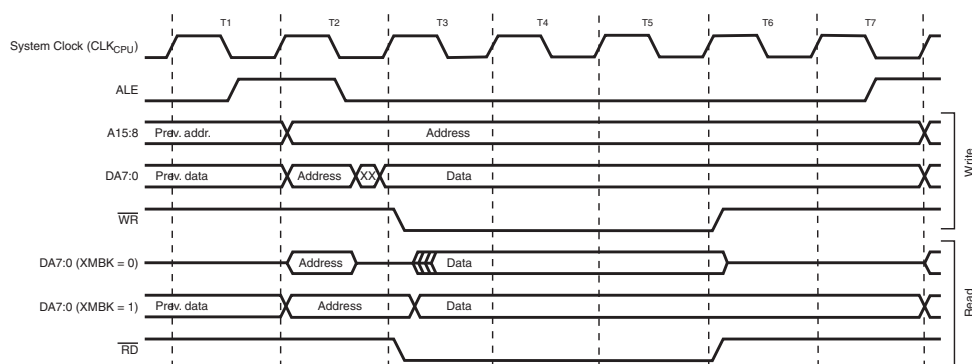
Note: 1. $SRWn1 = SRW11$ (upper sector) or $SRW01$ (lower sector), $SRWn0 = SRW10$ (upper sector) or $SRW00$ (lower sector)
The ALE pulse in period T5 is only present if the next instruction accesses the RAM (internal or external).

Figure 15. External Data Memory Cycles with $SRWn1 = 1$ and $SRWn0 = 0$ ⁽¹⁾



Note: 1. $SRWn1 = SRW11$ (upper sector) or $SRW01$ (lower sector), $SRWn0 = SRW10$ (upper sector) or $SRW00$ (lower sector).
The ALE pulse in period T6 is only present if the next instruction accesses the RAM (internal or external).

Figure 16. External Data Memory Cycles with SRWn1 = 1 and SRWn0 = 1⁽¹⁾



Note: 1. SRWn1 = SRW11 (upper sector) or SRW01 (lower sector), SRWn0 = SRW10 (upper sector) or SRW00 (lower sector).
The ALE pulse in period T7 is only present if the next instruction accesses the RAM (internal or external).

XMEM Register Description

MCU Control Register – MCUCR

Bit	7	6	5	4	3	2	1	0	
	SRE	SRW10	SE	SM1	ISC11	ISC10	ISC01	ISC00	MCUCR
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

• Bit 7 – SRE: External SRAM/XMEM Enable

Writing SRE to one enables the External Memory Interface. The pin functions AD7:0, A15:8, ALE, WR, and RD are activated as the alternate pin functions. The SRE bit overrides any pin direction settings in the respective Data Direction Registers. Writing SRE to zero, disables the External Memory Interface and the normal pin and data direction settings are used.

• Bit 6 – SRW10: Wait State Select Bit

For a detailed description, see common description for the SRWn bits below (EMUCR description).

Extended MCU Control Register – EMCUCR

Bit	7	6	5	4	3	2	1	0	
	SM0	SRL2	SRL1	SRL0	SRW01	SRW00	SRW11	ISC2	EMUCR
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

• Bit 6..4 – SRL2, SRL1, SRL0: Wait State Sector Limit

It is possible to configure different wait-states for different external memory addresses. The external memory address space can be divided in two sectors that have separate wait-state bits. The SRL2, SRL1, and SRL0 bits select the splitting of these sectors, see [Table 2](#) and [Figure 11](#). By default, the SRL2, SRL1, and SRL0 bits are set to zero and the entire external memory address space is treated as one sector. When the entire SRAM address space is configured as one sector, the wait-states are configured by the SRW11 and SRW10 bits.

Table 2. Sector Limits with Different Settings of SRL2..0

SRL2	SRL1	SRL0	Sector Limits
0	0	0	Lower sector = N/A Upper sector = 0x1100 - 0xFFFF
0	0	1	Lower sector = 0x1100 - 0x1FFF Upper sector = 0x2000 - 0xFFFF
0	1	0	Lower sector = 0x1100 - 0x3FFF Upper sector = 0x4000 - 0xFFFF
0	1	1	Lower sector = 0x1100 - 0x5FFF Upper sector = 0x6000 - 0xFFFF
1	0	0	Lower sector = 0x1100 - 0x7FFF Upper sector = 0x8000 - 0xFFFF
1	0	1	Lower sector = 0x1100 - 0x9FFF Upper sector = 0xA000 - 0xFFFF
1	1	0	Lower sector = 0x1100 - 0xBFFF Upper sector = 0xC000 - 0xFFFF
1	1	1	Lower sector = 0x1100 - 0xDFFF Upper sector = 0xE000 - 0xFFFF

• **Bit 1 and Bit 6 MCUCR – SRW11, SRW10: Wait-state Select Bits for Upper Sector**

The SRW11 and SRW10 bits control the number of wait-states for the upper sector of the external memory address space, see [Table 3](#).

• **Bit 3..2 – SRW01, SRW00: Wait-state Select Bits for Lower Sector**

The SRW01 and SRW00 bits control the number of wait-states for the lower sector of the external memory address space, see [Table 3](#).

Table 3. Wait-states⁽¹⁾

SRWn1	SRWn0	Wait-states
0	0	No wait-states
0	1	Wait one cycle during read/write strobe
1	0	Wait two cycles during read/write strobe
1	1	Wait two cycles during read/write and wait one cycle before driving out new address

Note: 1. n = 0 or 1 (lower/upper sector).

For further details of the timing and wait-states of the External Memory Interface, see [Figure 13](#) to [Figure 16](#) how the setting of the SRW bits affects the timing.

Special Function IO Register – SFIOR

Bit	7	6	5	4	3	2	1	0	
	TSM	XMBK	XMM2	XMM1	XMM0	PUD	PSR2	PSR310	SFIOR
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

• Bit 6 – XMBK: External Memory Bus Keeper Enable

Writing XMBK to one enables the Bus Keeper on the AD7:0 lines. When the Bus Keeper is enabled, AD7:0 will keep the last driven value on the lines even if the XMEM interface has tri-stated the lines. Writing XMBK to zero disables the Bus Keeper. XMBK is not qualified with SRE, so even if the XMEM interface is disabled, the bus keepers are still activated as long as XMBK is one.

• Bit 6..3 – XMM2, XMM1, XMM0: External Memory High Mask

When the External Memory is enabled, all Port C pins are used for the high address byte by default. If the full 60KB address space is not required to access the external memory, some, or all, Port C pins can be released for normal Port Pin function as described in [Table 4](#). As described in [“Using all 64KB Locations of External Memory” on page 34](#), it is possible to use the XMMn bits to access all 64KB locations of the external memory.

Table 4. Port C Pins Released as Normal Port Pins when the External Memory is Enabled

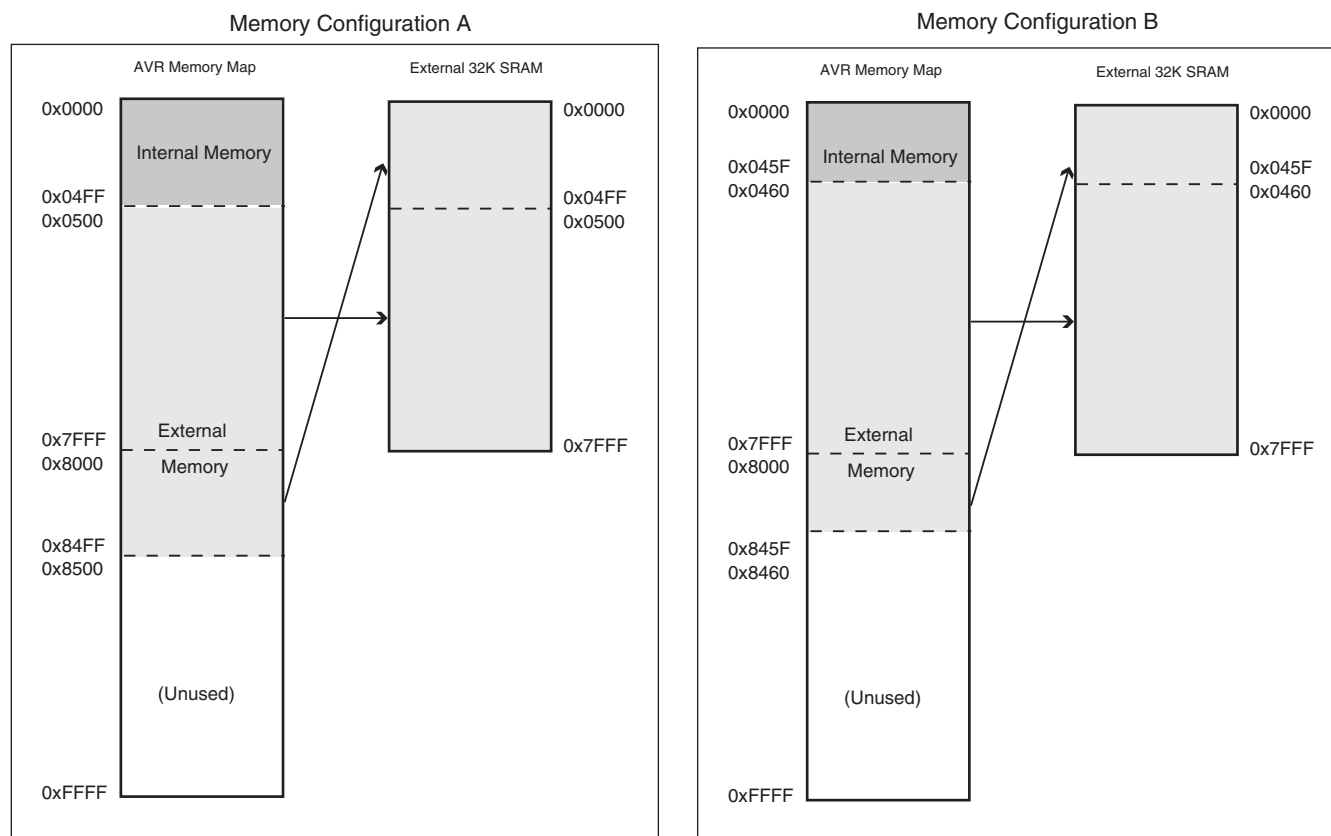
XMM2	XMM1	XMM0	# Bits for External Memory Address	Released Port Pins
0	0	0	8 (Full 60 KB space)	None
0	0	1	7	PC7
0	1	0	6	PC7 - PC6
0	1	1	5	PC7 - PC5
1	0	0	4	PC7 - PC4
1	0	1	3	PC7 - PC3
1	1	0	2	PC7 - PC2
1	1	1	No Address high bits	Full Port C

Using all Locations of External Memory Smaller than 64 KB

Since the external memory is mapped after the internal memory as shown in [Figure 11](#), the external memory is not addressed when addressing the first 1,280 bytes of data space. It may appear that the first 1,280 bytes of the external memory are inaccessible (external memory addresses 0x0000 to 0x04FF). However, when connecting an external memory smaller than 64 KB, for example 32 KB, these locations are easily accessed simply by addressing from address 0x8000 to 0x84FF. Since the External Memory Address bit A15 is not connected to the external memory, addresses 0x8000 to 0x84FF will appear as addresses 0x0000 to 0x04FF for the external memory. Addressing above address 0x84FF is not recommended, since this will address an external memory location that is already accessed by another (lower) address. To the Application software, the external 32 KB memory will appear as one linear 32 KB address space from 0x0500 to 0x84FF. This is illustrated in [Figure 17](#). Memory configuration B refers to the ATmega161 compatibility mode, configuration A to the non-compatible mode.

When the device is set in ATmega161 compatibility mode, the internal address space is 1,120 bytes. This implies that the first 1,120 bytes of the external memory can be accessed at addresses 0x8000 to 0x845F. To the Application software, the external 32 KB memory will appear as one linear 32 KB address space from 0x0460 to 0x845F.

Figure 17. Address Map with 32 KB External Memory



Using all 64KB Locations of External Memory

Since the external memory is mapped after the internal memory as shown in [Figure 11](#), only 64,256 Bytes of external memory are available by default (address space 0x0000 to 0x04FF is reserved for internal memory). However, it is possible to take advantage of the entire external memory by masking the higher address bits to zero. This can be done by using the XMMn bits and control by software the most significant bits of the address. By setting Port C to output 0x00, and releasing the most significant bits for normal Port Pin operation, the Memory Interface will address 0x0000 - 0x1FFF. See code example below.

Assembly Code Example⁽¹⁾

```
; OFFSET is defined to 0x2000 to ensure
; external memory access
; Configure Port C (address high byte) to
; output 0x00 when the pins are released
; for normal Port Pin operation

ldi r16, 0xFF
out DDRC, r16
ldi r16, 0x00
out PORTC, r16
; release PC7:5
ldi r16, (1<<XMM1) | (1<<XMM0)
out SFIOR, r16
; write 0xAA to address 0x0001 of external
; memory
ldi r16, 0xaa
sts 0x0001+OFFSET, r16
; re-enable PC7:5 for external memory
ldi r16, (0<<XMM1) | (0<<XMM0)
out SFIOR, r16
; store 0x55 to address (OFFSET + 1) of
; external memory
ldi r16, 0x55
sts 0x0001+OFFSET, r16
```

C Code Example⁽¹⁾

```
#define OFFSET 0x2000

void XRAM_example(void)
{
    unsigned char *p = (unsigned char *) (OFFSET + 1);

    DDRC = 0xFF;
    PORTC = 0x00;

    SFIOR = (1<<XMM1) | (1<<XMM0);

    *p = 0xaa;

    SFIOR = 0x00;

    *p = 0x55;
}
```

Note: 1. The example code assumes that the part specific header file is included.

Care must be exercised using this option as most of the memory is masked away.